

## Ejercicio 4: Conectividad al Mínimo Coste con Pesos Negativos

### Enunciado

El problema de “calcular un subconjunto  $A'$  de las aristas de un grafo con pesos, no dirigido y conexo  $G = (V, E)$  de forma que todos los vértices de  $G$  sigan estando conectados si se consideran únicamente las aristas de  $A'$  y que la suma de los pesos de las aristas de  $A'$  sea mínima” tiene sentido también aunque los pesos puedan ser negativos.

Sin embargo, la solución puede no ser un árbol (como ocurre en el caso de pesos no negativos, con el árbol de recubrimiento de peso mínimo). Se trata de adaptar el algoritmo de Kruskal o el de Prim para que funcione también para grafos que puedan tener pesos negativos.

### 1. Análisis Teórico y Observación Clave

En el problema clásico de Árbol de Recubrimiento Mínimo (MST) con pesos no negativos ( $w(e) \geq 0$ ), la solución óptima es siempre un árbol. Si existiera un ciclo, podríamos eliminar la arista de mayor peso de dicho ciclo, manteniendo el grafo conexo y reduciendo (o dejando igual) el coste total.

Sin embargo, cuando permitimos **pesos negativos o nulos** ( $w(e) \leq 0$ ), la situación cambia drásticamente:

- Si tenemos una arista con peso negativo  $w(e) < 0$  que forma parte de un ciclo, **no queremos eliminarla**. Al contrario: incluirla en nuestro conjunto  $A'$  reduce el coste acumulado de la solución y nunca perjudica la conectividad del grafo.
- De hecho, para cualquier arista  $e$  con  $w(e) \leq 0$ , **siempre es óptimo incluirla en  $A'$** . Añadir una arista de coste negativo o cero solo puede disminuir o mantener el coste total, y expande la conectividad del subgrafo.

### Estrategia de Resolución (Reducción por Contracción)

El problema de encontrar el subconjunto óptimo  $A'$  se puede resolver en dos fases lógicas:

1. **Fase 1 (Aristas Negativas/Nulas):** Seleccionamos obligatoriamente todas las aristas con peso  $w(e) \leq 0$ . Al añadirlas, los vértices del grafo se agruparán en un conjunto de componentes conexas (supernodos).
2. **Fase 2 (Aristas Positivas):** Para conectar estas componentes entre sí al mínimo coste, debemos seleccionar un conjunto de aristas positivas ( $w(e) > 0$ ) que no formen ciclos entre las componentes. Esto equivale exactamente a calcular un **Árbol de Recubrimiento Mínimo sobre el grafo contraído** por las componentes de la Fase 1.

### 2. Adaptación del Algoritmo de Kruskal

El algoritmo de Kruskal clásico utiliza una estructura de partición (conjuntos disjuntos o *Union-Find*) para evitar ciclos. Adaptarlo para este problema es sumamente sencillo y elegante:

1. **Inicialización:** Creamos un conjunto vacío  $A'$  para almacenar las aristas elegidas. Inicializamos la estructura *Union-Find* con todos los vértices del grafo por separado.
2. **Paso 1 (Fuerza Bruta para Negativos):** Recorremos todas las aristas del grafo. Para cada arista  $e = (u, v)$  con  $w(e) \leq 0$ :
  - La añadimos incondicionalmente a  $A'$ :  $A' = A' \cup \{e\}$ .
  - Unimos las componentes conexas de sus extremos en el Union-Find:  $\text{Union}(u, v)$ .
3. **Paso 2 (Kruskal Estándar para Positivos):**
  - Filtramos y ordenamos las aristas restantes (aquellas con  $w(e) > 0$ ) en orden creciente de peso.
  - Recorremos estas aristas positivas ordenadas de menor a mayor. Para cada arista  $e = (u, v)$ :
    - Si  $\text{Find}(u) \neq \text{Find}(v)$  (es decir, pertenecen a componentes conexas distintas):
      - Añadimos la arista a  $A'$ .
      - Unimos sus componentes:  $\text{Union}(u, v)$ .
    - Si ya pertenecían a la misma componente conexa, descartamos la arista (ya que añadir una arista positiva que genera un ciclo solo aumentaría el coste sin aportar conectividad necesaria).

### 3. Algoritmo en Pseudocódigo

```
Algoritmo KruskalModificado(V, E)
  // Entrada:
  //   - V: Conjunto de vértices del grafo
  //   - E: Conjunto de aristas con sus pesos w(e)
  // Salida:
  //   - A_prime: Conjunto óptimo de aristas que mantiene conectado el grafo al

  A_prime ← ∅ // Conjunto vacío para la solución

  // 1. Inicializar estructura Union-Find para los |V| vértices
  Para cada v en V hacer
    MakeSet(v)
  FinPara

  Aristas_Positivas ← ∅

  // 2. Procesar aristas negativas y nulas de forma incondicional
  Para cada e = (u, v) en E hacer
    Si w(e) ≤ 0 entonces
      A_prime ← A_prime ∪ {e}
      Union(u, v)
    Sino
      Aristas_Positivas ← Aristas_Positivas ∪ {e}
    FinSi
  FinPara
```

```

// 3. Ordenar las aristas positivas de menor a mayor peso
Ordenar_Ascendente_Por_Peso(Aristas_Positivas)

// 4. Aplicar Kruskal clásico sobre las aristas positivas para unir component
Para cada e = (u, v) en Aristas_Positivas hacer
    Si Find(u) ≠ Find(v) entonces
        A_prime ← A_prime U {e}
        Union(u, v)
    FinSi
FinPara

Retornar A_prime
FinAlgoritmo

```

## 4. Justificación de Correctitud y Complejidad

### Demostración de Optimalidad

- **Inclusión de  $w(e) \leq 0$ :** Supongamos que existe una solución óptima  $S$  que no incluye una arista  $e' = (u, v)$  con  $w(e') \leq 0$ . Si añadimos  $e'$  a  $S$ , la conectividad del grafo se mantiene (o mejora), y el coste total de la solución será  $\text{coste}(S) + w(e') \leq \text{coste}(S)$ . Por tanto, añadir aristas no positivas nunca empeora la solución. Podemos incluirlas todas desde el principio de forma segura.
- **Selección de  $w(e) > 0$ :** Una vez añadidas las aristas no positivas, el grafo queda dividido en  $C \geq 1$  componentes conexas. Cualquier solución válida debe conectar estas componentes usando aristas positivas. Para minimizar el coste, debemos seleccionar el conjunto de aristas positivas de menor peso acumulado que elimine la desconexión entre componentes, lo cual se reduce exactamente al problema de encontrar el Árbol de Recubrimiento Mínimo (MST) de las componentes utilizando las aristas positivas. Kruskal clásico es óptimo para esta tarea.

### Análisis de Coste

- **Estructura Union-Find:** Las operaciones `MakeSet`, `Union` y `Find` con optimización de rango y compresión de caminos tienen un coste prácticamente constante  $\mathcal{O}(\alpha(|V|))$ .
- **Fase de Negativos:** Recorrer las aristas y aplicar `Union` para las negativas cuesta  $\mathcal{O}(|E| \cdot \alpha(|V|))$ .
- **Fase de Ordenación:** Ordenar las aristas positivas de tamaño  $k \leq |E|$  cuesta  $\mathcal{O}(|E| \log |E|)$  (que equivale a  $\mathcal{O}(|E| \log |V|)$  en grafos conexos simples).
- **Fase de Kruskal:** Recorrer y aplicar las uniones restantes cuesta  $\mathcal{O}(|E| \cdot \alpha(|V|))$ .
- **Coste Total:**  $\mathcal{O}(|E| \log |V|)$  en tiempo. No hay ninguna penalización temporal por la presencia de pesos negativos en comparación con el algoritmo clásica de Kruskal.